14. 검증자 보안성 분석 및 암호학적 가정

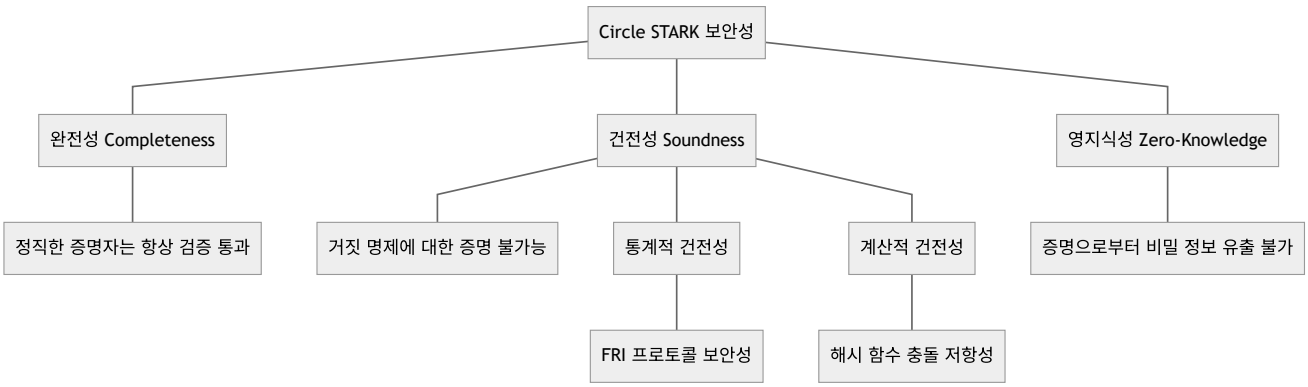
개요

Stwo의 Circle STARK 시스템은 여러 암호학적 가정과 보안 메커니즘을 기반으로 구축되어 있습니다. 이 문서는 시스템의 보안성 근거, 암호학적 가정, 알려진 공격 방법과 대응 방안, 그리고 보안 매개변수 선택에 대해 상세히 분석합니다.

1. Circle STARK의 핵심 보안성 근거

1.1 암호학적 보안 모델

Circle STARK의 보안성은 다음과 같은 핵심 속성들에 기반합니다:



1.2 보안성 정의

완전성 (Completeness):

- 정직한 증명자가 참인 명제에 대해 생성한 증명은 항상 검증을 통과
- 확률: 1 (결정론적)

건전성 (Soundness):

- 거짓 명제에 대해 악의적 증명자가 검증을 통과할 확률이 무시할 수 있을 정도로 작음
- 확률: 2^(-λ) (λ는 보안 매개변수)

영지식성 (Zero-Knowledge):

- 증명으로부터 원본 데이터에 대한 정보를 추출할 수 없음
- 시뮬레이터 존재성으로 보장

2. 암호학적 가정 분석

2.1 해시 함수 보안성 가정

Stwo는 두 가지 주요 해시 함수를 지원합니다:

2.1.1 Blake2s 해시 함수

```
// Blake2s 채널 구현
pub struct Blake2sChannel {
digest: Blake2sHash,
pub channel_time: ChannelTime,
}

impl Channel for Blake2sChannel {
const BYTES_PER_HASH: usize = 32; // 256비트 출력
fn trailing_zeros(&self) -> u32 {
```

```
u128::from_le_bytes(array::from_fn(|i| self.digest.0[i])).trailing_zeros()
}
}
```

Blake2s 보안 가정:

- **충돌 저항성**: 2^128 보안 수준
- **원상 저항성**: 2^256 보안 수준
- **제2 원상 저항성**: 2^256 보안 수준
- **의사 무작위성**: 구별 불가능한 의사 무작위 함수

2.1.2 Poseidon252 해시 함수

```
// Poseidon252 채널 구현
pub struct Poseidon252Channel {
digest: FieldElement252,
pub channel_time: ChannelTime,
}

impl Channel for Poseidon252Channel {
const BYTES_PER_HASH: usize = 31; // 252비트 출력 (31바이트)
fn trailing_zeros(&self) -> u32 {
let bytes = self.digest.to_bytes_be();
u128::from_be_bytes(bytes[16..].try_into().unwrap()).trailing_zeros()
}
```

```
}
```

Poseidon252 보안 가정:

- **대수적 해시 함수**: 유한체 연산 기반
- **STARK 친화적**: 제약 조건 수가 적음
- **충돌 저항성**: 2^126 보안 수준 (252비트 필드)
- **대수적 공격 저항성**: Gröbner 기저 공격 등에 대한 저항성

2.2 필드 연산 보안성 가정

2.2.1 M31 기본 필드

```
// M31 필드: 2^31 - 1
pub const P: u32 = (1 << 31) - 1;

impl BaseField {
pub const fn reduce(x: u64) -> Self {
// 모듈러 연산을 통한 필드 원소 생성
Self::from_u32_unchecked((x % P as u64) as u32)
}
}
```

M31 보안 가정:

- **이산 로그 문제**: 2^31 크기 필드에서의 이산 로그 어려움
- **유한체 연산 정확성**: 모듈러 연산의 수학적 정확성

- **하드웨어 최적화**: 32비트 연산의 효율성

2.2.2 QM31 확장 필드

```
// QM31: M31의 4차 확장 필드
pub const SECURE_EXTENSION_DEGREE: usize = 4;

pub struct SecureField {
// 4개의 M31 원소로 구성
pub a: BaseField,
pub b: BaseField,
pub c: BaseField,
pub d: BaseField,
}
```

QM31 보안 가정:

- **확장 필드 보안성**: 2^124 보안 수준 (31 × 4비트)
- **기약 다항식**: 확장 필드 구성의 수학적 정확성
- **복소수 켤레**: Circle 기하학과의 호환성

3. FRI 프로토콜 보안성 분석

3.1 FRI 보안 매개변수

```
#[derive(Debug, Clone, Copy)]
```

```rust
pub struct FriConfig {
pub log_blowup_factor: u32, // 블로우업 인수
pub log_last_layer_degree_bound: u32, // 마지막 레이어 차수 경계
pub n_queries: usize, // 쿼리 수
}

impl FriConfig {
pub const fn security_bits(&self) -> u32 {
self.log_blowup_factor * self.n_queries as u32
}
}
```

3.2 FRI 건전성 분석

건전성 오류 확률:

```
P_error ≤ (1 - ρ)^t + 2^(-λ)
```

여기서:

- ρ = 1/blowup_factor (근접성 매개변수)
- t = n_queries (쿼리 수)
- λ = 보안 매개변수

일반적인 매개변수 설정:

```rust
// 예시: 128비트 보안 수준
```

```
let  config = FriConfig {
log_blowup_factor: 2, // 4배 블로우업
log_last_layer_degree_bound: 5, // 32차 다항식
n_queries: 64, // 64개 쿼리
};
// 보안 수준: 2 * 64 = 128비트
```

3.3 Circle 도메인 특수성

Circle FFT 보안성:

- **복소수 원 기하학**: 전통적인 승법군 대신 복소수 원 사용
- **점 배가 알고리즘**: 효율적이고 안전한 점 연산
- **부분군 구조**: 수학적으로 검증된 구조

```
// Circle 포인트 보안성
impl CirclePoint<SecureField> {
pub  fn  get_random_point(channel: &mut  impl Channel) -> Self {
// Fiat-Shamir 변환을 통한 안전한 무작위 포인트 생성
let  x = channel.draw_secure_felt();
let  y_squared = SecureField::one() - x * x;
let  y = y_squared.sqrt().unwrap_or(SecureField::zero());
Self { x, y }
}
```

```
}
```

4. 알려진 공격 방법과 대응 방안

4.1 대수적 공격 (Algebraic Attacks)

공격 방법:

- 다항식 시간 내에 제약 조건을 만족하는 거짓 증명 생성 시도
- Gröbner 기저를 이용한 방정식 시스템 해결

대응 방안:

```
// 무작위 계수를 통한 선형 결합
let  random_coeff = channel.draw_secure_felt();
let  composition_eval = components.eval_composition_polynomial_at_point(
oods_point,
&proof.sampled_values,
random_coeff, // 예측 불가능한 무작위 계수
);
```

4.2 FRI 근접성 공격 (Proximity Attacks)

공격 방법:

- 낮은 차수 다항식에 가깝지만 실제로는 높은 차수인 함수 구성
- 제한된 쿼리로 검출되지 않는 거짓 증명 생성

대응 방안:

```
// 충분한 쿼리 수와 블로우업 인수 설정
impl FriConfig {
const LOG_MIN_BLOWUP_FACTOR: u32 = 1; // 최소 2배 블로우업
const LOG_MAX_BLOWUP_FACTOR: u32 = 16; // 최대 65536배 블로우업
pub fn new(log_last_layer_degree_bound: u32, log_blowup_factor: u32, n_queries
assert!(Self::LOG_BLOWUP_FACTOR_RANGE.contains(&log_blowup_factor));
// 보안 매개변수 검증
}
}
```

4.3 해시 충돌 공격 (Hash Collision Attacks)

공격 방법:

- Merkle 트리에서 해시 충돌을 이용한 거짓 증명 생성
- 서로 다른 데이터에 대해 동일한 해시값 생성

대응 방안:

```
// 강력한 해시 함수 사용 및 충돌 검증
impl<H: MerkleHasher> MerkleVerifier<H> {
pub fn verify(&self, /* ... */) -> Result<(), MerkleVerificationError> {
// 계산된 루트와 약속된 루트 비교
if computed_root != self.root {
return Err(MerkleVerificationError::RootMismatch);
```

```
}
Ok(())
}
}
```

4.4 Grinding 공격 (Proof of Work Bypass)

공격 방법:

- 작업 증명 요구사항을 우회하여 저비용으로 거짓 증명 생성
- 병렬 처리를 통한 무차별 대입 공격

대응 방안:

```
// 작업 증명 검증
fn verify_proof_of_work(channel: &impl Channel, proof_of_work: u64, pow_bits: u
channel.mix_u64(proof_of_work);
channel.trailing_zeros() >= pow_bits
}

// 적응적 난이도 조절
pub struct PcsConfig {
pub pow_bits: u32, // 작업 증명 난이도
pub fri_config: FriConfig,
}
```

5. 보안 매개변수 선택 가이드라인

5.1 보안 수준별 권장 매개변수

5.1.1 80비트 보안 수준 (테스트/개발용)

```
let  config_80bit = PcsConfig {
pow_bits: 10, // 2^10 = 1024 해시 연산
fri_config: FriConfig::new(
4, // 16차 마지막 레이어
1, // 2배 블로우업
40, // 40개 쿼리 (1 * 40 = 40비트)
),
};
```

5.1.2 128비트 보안 수준 (프로덕션 권장)

```
let  config_128bit = PcsConfig {
pow_bits: 15, // 2^15 = 32768 해시 연산
fri_config: FriConfig::new(
5, // 32차 마지막 레이어
2, // 4배 블로우업
64, // 64개 쿼리 (2 * 64 = 128비트)
),
```

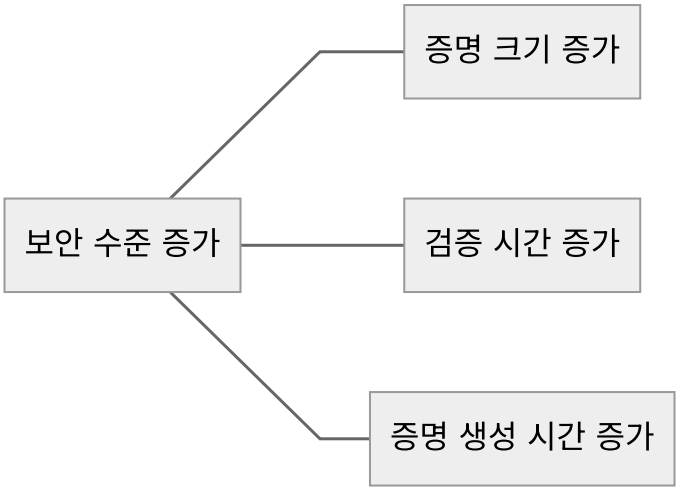
```
};
```

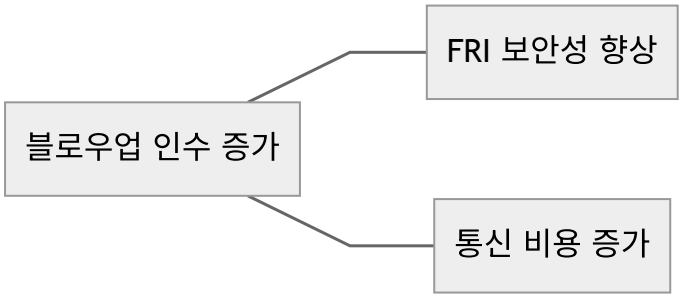
5.1.3 256비트 보안 수준 (최고 보안)

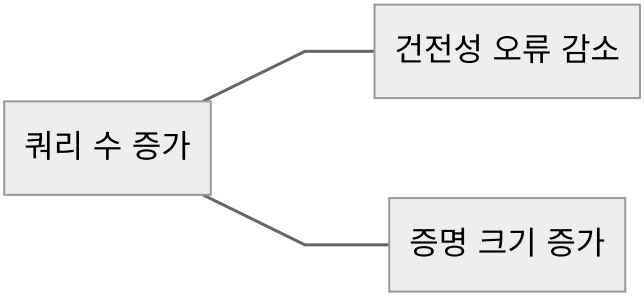
```
let  config_256bit = PcsConfig {
pow_bits: 20, // 2^20 = 1048576 해시 연산
fri_config: FriConfig::new(
6, // 64차 마지막 레이어
3, // 8배 블로우업
128, // 128개 쿼리 (3 * 128 = 384비트 > 256비트)
),
};
```

5.2 매개변수 선택 고려사항

5.2.1 성능 vs 보안 트레이드오프







5.2.2 애플리케이션별 권장사항

블록체인 애플리케이션:

- 높은 보안 수준 필요 (128비트 이상)
- 검증 비용 최적화 중요
- 배치 검증 활용 권장

영지식 증명 서비스:

- 매우 높은 보안 수준 (256비트)
- 증명 크기 최적화 중요

- 전용 하드웨어 활용 고려

연구/개발 환경:

- 중간 보안 수준 (80-128비트)
- 빠른 반복 개발 중요
- 디버깅 기능 활용

6. 보안성 검증 및 감사

6.1 형식적 검증 (Formal Verification)

수학적 증명:

- FRI 프로토콜의 건전성 증명
- Circle FFT의 정확성 증명
- 확률적 오류 경계 분석

구현 검증:

```
#[cfg(test)]
mod security_tests {
use super::*;
#[test]
fn test_soundness_error_bound() {
let config = FriConfig::new(5, 2, 64);
let theoretical_security = config.security_bits();
assert!(theoretical_security >= 128);
}
```

```rust
#[test]
fn test_hash_collision_resistance() {
// 해시 충돌 저항성 테스트
let mut channel = Blake2sChannel::default();
let hash1 = channel.draw_random_bytes();
let hash2 = channel.draw_random_bytes();
assert_ne!(hash1, hash2);
}
}
```

6.2 보안 감사 체크리스트

암호학적 구성요소:

- [ ] 해시 함수 보안성 검증
- [ ] 필드 연산 정확성 확인
- [ ] 무작위성 생성 품질 검사
- [ ] Fiat-Shamir 변환 구현 검증

프로토콜 구현:

- [ ] FRI 매개변수 유효성 검사
- [ ] 에러 처리 완전성 확인
- [ ] 사이드 채널 공격 저항성 검사
- [ ] 메모리 안전성 검증

시스템 통합:

- [ ] 전체 증명 시스템 일관성 확인
- [ ] 성능 vs 보안 균형 검증
- [ ] 확장성 및 유지보수성 평가

7. 미래 보안 고려사항

7.1 양자 컴퓨팅 위협

현재 상황:

- Circle STARK는 양자 컴퓨터에 대해 부분적 저항성 보유
- 해시 함수는 Grover 알고리즘에 의해 보안 수준 절반 감소

대응 방안:

- 보안 매개변수 2배 증가 (128비트 → 256비트)
- 양자 저항성 해시 함수 연구 및 도입
- 격자 기반 암호학과의 결합 고려

7.2 새로운 공격 기법

지속적 모니터링:

- 학술 연구 동향 추적
- 새로운 대수적 공격 기법 분석
- 하드웨어 발전에 따른 보안 수준 재평가

적응적 보안:

- 매개변수 업그레이드 메커니즘
- 하위 호환성 유지 방안
- 긴급 보안 패치 프로세스

결론

Stwo의 Circle STARK 시스템은 견고한 암호학적 기반 위에 구축되어 있으며, 다층적인 보안 메커니즘을 통해 높은 수준의 보안성을 제공합니다. 적절한 매개변수 선택과 지속적인 보안 감사를 통해 현재와 미래의 위협에 대응할 수 있는 강력한 영지식 증명 시스템을 구현할 수 있습니다.